OWASP Juice Shop – Penetration Testing Writeup

Target: OWASP Juice Shop (TryHackMe Lab)
Application URL: http://10.114.140.87
Assessment Type: Web Application Penetration Test
Testing Approach: Grey Box
Report Date: 2025

> **Confidentiality Notice:** This report contains sensitive, privileged, and confidential information intended solely for authorized parties. Publication may cause reputational damage or facilitate attacks against the assessed system.

> **Disclaimer:** This assessment may not disclose all vulnerabilities present. It is a point-in-time assessment; changes to the environment during testing may affect results.

Table of Contents

Executive Summary

A penetration test was conducted against the OWASP Juice Shop web application. The assessment simulated a real-world attacker targeting the application with the goal of identifying and exploiting vulnerabilities across multiple categories including injection, broken authentication, sensitive data exposure, broken access control, and cross-site scripting (XSS).

A total of **15 vulnerabilities** were identified and exploited during the engagement.

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	5	6	3	1

The most severe findings include SQL Injection enabling full administrative account takeover, stored/reflected/DOM-based XSS vulnerabilities, unauthenticated access to sensitive FTP files, and broken access control allowing horizontal privilege escalation into other users' shopping baskets. **Immediate remediation of injection and authentication flaws is strongly recommended.**

Findings Overview

#	Finding	Risk Score	Risk Level
1	SQL Injection – Admin Login Bypass	9	**High**
2	SQL Injection – Account Takeover (Bender)	8	**High**
3	Broken Authentication – Weak Admin Password	8	**High**
4	Persistent XSS via HTTP Header (True-Client-IP)	8	**High**
5	Broken Access Control – Admin Panel Exposed	7	**High**
6	Reflected XSS via Order Tracking ID	6	Medium
7	DOM XSS via Search Bar	6	Medium
8	IDOR – View Other Users' Baskets	6	Medium
9	Sensitive Data Exposure – Public /ftp/ Directory	6	Medium
10	Broken Authentication – Password Reset via Security Question	5	Medium
11	Broken Authentication – Credentials in Public Media	5	Medium
12	Poison Null Byte – File Extension Restriction Bypass	4	Medium

#	Finding	Risk Score	Risk Level
13	Information Disclosure – Admin Email via Product Page	3	Low
14	Information Disclosure – User Emails via Product Reviews	3	Low
15	Missing Access Control – Unauthorized Deletion of Reviews	2	Low

Reconnaissance & Methodology

Testing was conducted across three phases:

Phase 1 – Reconnaissance (Walking the Application)

Burp Suite was configured as a proxy with Intercept set to **off**. The application was browsed manually, allowing Burp to log all HTTP requests and responses. This technique — known as "walking the application" — revealed user emails, hidden endpoints, and application structure without active probing.

Key discoveries during recon:

- **Admin email** (`admin@juice-sh.op`) exposed on the Apple Juice product page
- **Jim's email** (`jim@juice-sh.op`) found in a Green Smoothie product review mentioning a "replicator" (a Star Trek reference)
- **Bender's email** (`bender@juice-sh.op`) found in a Banana Juice product review
- **MC SafeSearch's email** (`mc.safesearch@juice-sh.op`) found on the Juice Shop Permafrost 2020 Edition product page

Phase 2 – Target Assessment

JavaScript source files (`main-es2015.js`) were analyzed using Firefox DevTools to discover hidden routes such as `/#/administration`. The `/ftp/` directory was enumerated directly. Product reviews, error messages, and HTTP response headers were analyzed for information disclosure.

Phase 3 – Exploitation

Identified vulnerabilities were exploited to demonstrate real-world impact, including:

- SQL injection via Burp Suite's Intercept for account takeover
- XSS payload injection across three different vectors
- Directory traversal via Poison Null Byte bypass
- Broken access control via manipulated API basket ID parameter

Findings

---

1. SQL Injection – Admin Login Bypass

Attribute	Value
Risk Level	🔴 HIGH (9/10)
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications
An attacker can log into the administrator account without knowing the password by injecting a SQL payload into the login form, granting full administrative access to the application.

Steps to Reproduce

1. Navigate to the login page: `http://10.114.140.87/#/login`
2. Open Burp Suite and enable **Intercept**
3. Enter any value in the email and password fields, then click Login
4. In Burp, change the email field value to: `' or 1=1--`
5. Forward the request — you will be authenticated as the admin user

The backend SQL query becomes:

```sql
SELECT * FROM users WHERE email='' or 1=1--' AND password='...'
```

The `or 1=1` always evaluates to true, and `--` comments out the rest of the query, bypassing authentication entirely.

Flag: `690fa3247a99d651e0b26f947baf0b79b4f404a9`

Recommendations

- Use parameterized queries / prepared statements for all database interactions
- Implement input validation and server-side sanitization on all login fields
- Deploy a Web Application Firewall to detect SQL injection patterns

References: https://owasp.org/www-community/attacks/SQL_Injection

2. SQL Injection – Account Takeover (Bender)

Attribute	Value
Risk Level	🔴 HIGH (8/10)
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications

Any registered user account can be hijacked without a password using a SQL comment-based injection on the login page.

Steps to Reproduce

1. Obtain the target email — `bender@juice-sh.op` was found in the Banana Juice product review
2. Navigate to the login page
3. Enable Burp Intercept and submit a login request
4. Change the email field to: `bender@juice-sh.op'--`
5. Forward — this comments out the password check and logs in as Bender

The injected query becomes:

```sql
SELECT * FROM users WHERE email='bender@juice-sh.op'--' AND password='...'
```

Everything after `--` is treated as a comment, so the password is never checked.

Flag: `5ff5052e879e6fef64124e64c82c84ebc809c6c4`

Recommendations

- Use parameterized queries / prepared statements for all database interactions
- Enforce server-side input validation on all authentication endpoints

3. Broken Authentication – Weak Admin Password

Attribute	Value
Risk Level	🔴 HIGH (8/10)
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications

The administrator account uses a trivially guessable password (`admin123`), allowing unauthorized access via brute force or simple guessing without any bypass techniques.

Steps to Reproduce

1. Navigate to the login page
2. Enter email: `admin@juice-sh.op` (discovered via the Apple Juice product page)
3. Enter password: `admin123`
4. Login is successful — no SQL injection or bypass required

The application does not enforce password complexity policies or account lockout after repeated failed attempts.

Flag: `ff4aebffe31b0ffdea9bdd0207a16a3c01ac6c56`

Recommendations

- Enforce a strong password policy (minimum 12 characters, mixed case, numbers, symbols)
- Implement account lockout after repeated failed login attempts
- Enable multi-factor authentication (MFA) for all privileged accounts

---

4. Persistent XSS via HTTP Header (True-Client-IP)

Attribute	Value
Risk Level	🔴 HIGH (8/10)
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Moderate

Security Implications
A stored XSS payload injected via the `True-Client-IP` HTTP header is rendered unsanitized on the Last Login IP page, executing malicious JavaScript for every admin viewing that page.

Steps to Reproduce

1. Log into the admin account
2. Open Burp Suite Intercept and click **Logout**
3. Intercept the logout request and add the following HTTP header:

```
True-Client-IP: <iframe src="javascript:alert(`xss`)">
```

4. Forward the request
5. Log back in as admin and navigate to **Account → Last Login IP**
6. The XSS payload executes — the iframe triggers a JavaScript alert

This is a **persistent (stored)** XSS because the injected value is saved server-side and executed on every page load.

Flag: `c37da14686b69a220fd9febd09bb9593e7d0539f`

Recommendations

- Sanitize and validate all HTTP request headers before storing or rendering them
- Implement a Content Security Policy (CSP) header to restrict script execution sources
- Encode all output rendered from user-controlled or header-sourced data

References: https://owasp.org/www-community/attacks/xss/

---

5. Broken Access Control – Admin Panel Exposed via JavaScript

Attribute	Value
Risk Level	🔴 HIGH (7/10)
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications

The administration panel route is embedded in client-side JavaScript, allowing any user who inspects the source to navigate to `/#/administration` and access admin functions.

Steps to Reproduce

1. Open Firefox DevTools → **Debugger** (or Chrome DevTools → **Sources**)
2. Refresh the page and locate `main-es2015.js`
3. Search for the keyword `admin`
4. Find the entry: `path: 'administration'`
5. Navigate to `http://10.114.140.87/#/administration` while logged in as admin
6. Full admin panel is accessible — user reviews, feedback, and management tools are exposed

Flag: `71aeb3b0bf01cc6e488f0207bb62f79b41454a87`

Recommendations

- Enforce server-side authorization checks on all admin API endpoints; do not rely on client-side route hiding
- Restrict the administration panel to specific IP ranges or require additional authentication
- Remove or obfuscate sensitive route information from bundled JavaScript files

---

6. Reflected XSS via Order Tracking ID

Attribute	Value
Risk Level	🟡 MEDIUM (6/10)
Exploitation Likelihood	Likely
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications

User-supplied input in the order tracking URL parameter is reflected in the page response without sanitization, enabling reflected XSS attacks against any user who clicks a crafted link.

Steps to Reproduce

1. Log in and navigate to **Order History**
2. Click the truck/tracking icon on any order

3. Note the tracking ID in the URL
4. Replace the tracking ID with: `<iframe src="javascript:alert(`xss`)">`
5. The XSS payload executes on page load — a JavaScript alert appears

Flag: `305021787d3e9cd9cebc057a021c2504550bb3b6`

Recommendations

- Sanitize and encode all URL parameters before including them in server responses or rendering them in the DOM
- Implement Content Security Policy (CSP) headers

---

7. DOM XSS via Search Bar

Attribute	Value
Risk Level	🟠 MEDIUM (6/10)
Exploitation Likelihood	Likely
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications
The application search bar passes user input directly into the DOM without sanitization, allowing DOM-based XSS payloads to execute in the browser.

Steps to Reproduce

1. Click the magnifying glass icon in the top right of the application
2. In the search bar, enter the following payload:

```html
<iframe src="javascript:alert(`xss`)">
```

3. The JavaScript alert fires — the input was parsed by the DOM and executed as code

The search parameter is visible in the URL as `/#/search?q=...`. The application uses unsafe DOM manipulation (likely `innerHTML`) to render results.

Flag: `4a31a4fe0954199566e360a873802bf64d0d0a84`

Recommendations

- Avoid using `innerHTML` or other unsafe DOM manipulation methods with user-supplied data
- Use safe DOM APIs such as `textContent` or `createTextNode`
- Implement a strict Content Security Policy (CSP) to prevent inline script execution

References: https://owasp.org/www-community/attacks/DOM_Based_XSS

---

8. IDOR – View Other Users' Baskets

Attribute	Value
Risk Level	🟠 MEDIUM (6/10)
Exploitation Likelihood	Likely
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications

The `/api/basket/{id}` endpoint does not validate that the requesting user owns the basket, allowing any authenticated user to view another user's shopping basket by manipulating the ID.

Steps to Reproduce

1. Log in as admin and navigate to **Your Basket**
2. Open Burp Suite and intercept the basket request
3. Observe the request: `GET /rest/basket/1`
4. Change the basket ID from `1` to `2`
5. Forward the request — another user's basket contents are returned with no authorization error

This is a classic **Insecure Direct Object Reference (IDOR)** — the application trusts the client-supplied basket ID without verifying ownership.

Flag: `e6982b34b6734ceadd28e5019b251f929a80b815`

Recommendations

- Implement server-side ownership checks on all object references
- Use indirect references (e.g., session-tied tokens) rather than predictable sequential IDs

---

9. Sensitive Data Exposure – Public /ftp/ Directory

Attribute	Value
Risk Level	🟡 MEDIUM (6/10)
Exploitation Likelihood	Likely
Business Impact	Major
Remediation Difficulty	Easy

Security Implications

The `/ftp/` directory is publicly accessible without authentication, exposing confidential business documents including an acquisitions file and developer backup files.

Steps to Reproduce

1. Navigate to the **About Us** page
2. Hover over "Check out our terms of use" — the link reveals `http://10.114.140.87/ftp/legal.md`
3. Navigate directly to `http://10.114.140.87/ftp/`
4. A full directory listing is revealed, containing files such as:
   - `legal.md`
   - `acquisitions.md` (confidential business document)
   - `package.json.bak` (developer backup)
5. Download `acquisitions.md` — returning to the home page triggers the challenge flag

Flag: `8d2072c6b0a455608ca1a293dc0c9579883fc6a5`

Recommendations

- Remove the `/ftp/` directory from public web server access immediately
- Implement access controls and authentication for any file-serving directories
- Audit all web-accessible directories for unintended file exposure

---

10. Broken Authentication – Password Reset via Weak Security Question (Jim)

Attribute	Value
Risk Level	🟡 MEDIUM (5/10)

Attribute	Value
Exploitation Likelihood	Likely
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications

Jim's account password can be reset by anyone who can research the answer to his security question, which references a publicly known pop culture fact.

Steps to Reproduce

1. Jim's email (`jim@juice-sh.op`) was found in a Green Smoothie product review — the review mentions a "replicator"
2. Googling "replicator" reveals it is a device from the TV show **Star Trek**
3. Navigate to **Forgot Password** and enter `jim@juice-sh.op`
4. The security question asks about his brother's middle name
5. Through OSINT (Star Trek character research), the answer is **Samuel**
6. Enter `Samuel` → set a new password (e.g., `Hacker_123`) → account is taken over

Flag: `3c3e2d6ef99b733b947e92f8e2a9ed08bf57ea63`

Recommendations

- Replace knowledge-based security questions with multi-factor authentication
- Implement rate limiting and CAPTCHA on the password reset endpoint

---

11. Broken Authentication – Credentials Embedded in Public Media

Attribute	Value
Risk Level	🟡 MEDIUM (5/10)
Exploitation Likelihood	Likely
Business Impact	Moderate
Remediation Difficulty	Easy

Security Implications

MC SafeSearch's account password is disclosed in a publicly available promotional video, allowing anyone who watches it to log into the account.

Steps to Reproduce

1. Navigate to the **Juice Shop Permafrost 2020 Edition** product page
2. The associated promotional video features MC SafeSearch stating his password is "Mr. Noodles" but with vowels replaced by zeros
3. Decode the hint: `o → 0`, giving the password: `Mr. N00dles`
4. Log in using:
   - Email: `mc.safesearch@juice-sh.op`
   - Password: `Mr. N00dles`

Flag: `bb105418e73708ceccf1a7b2491f434b8f5230e4`

Recommendations

- Never embed credentials or password hints in marketing or public-facing media
- Enforce periodic password changes and ensure credentials are not shared externally

---

12. Poison Null Byte – File Extension Restriction Bypass

Attribute	Value
Risk Level	🟡 MEDIUM (4/10)
Exploitation Likelihood	Possible
Business Impact	Moderate
Remediation Difficulty	Moderate

Security Implications

The `/ftp/` directory restricts file downloads to `.md` and `.pdf` extensions, but this control can be bypassed using a URL-encoded Poison Null Byte, allowing download of arbitrary files.

Steps to Reproduce

1. Navigate to `http://10.114.140.87/ftp/`
2. Attempt to download `package.json.bak` — receive a `403 Forbidden` error:

```
Only .md and .pdf files are allowed!
```

3. Modify the URL by appending a Poison Null Byte encoded for URL use:

```
http://10.114.140.87/ftp/package.json.bak%2500.md
```

- `%00` = null byte, `%25` = URL-encoded `%`, so `%2500` = URL-encoded null byte

4. The server is tricked into thinking the file ends in `.md` and allows the download
5. The `package.json.bak` developer backup file is successfully downloaded

Flags: `dad737329153068527159005db4eb139e86e76ba` | `cfdeea14e8f01b4952722fd0e4a77f1928593c9a`

Note: The Error Handling flag (`9c297196ecf8890bc1e900fcf3aebae8c9f9880a`) was also triggered during this process.

Recommendations

- Implement server-side file type validation using MIME type detection, not filename strings
- Sanitize and reject URLs containing null bytes or other special characters before processing
- Remove the `/ftp/` directory from public access entirely

References: https://owasp.org/www-community/attacks/Null_Byte_Injection

---

13. Information Disclosure – Admin Email via Product Page

Attribute	Value
Risk Level	🟢 LOW (3/10)
Exploitation Likelihood	Likely
Business Impact	Minor
Remediation Difficulty	Easy

Security Implications
The administrator's email address is displayed publicly on a product detail page, providing attackers with a valid username for brute force or social engineering attacks.

Steps to Reproduce

1. Navigate to the home page
2. Click on the **Apple Juice** product
3. View the product reviews section — `admin@juice-sh.op` is visible without authentication

Recommendations

- Avoid displaying internal or administrative email addresses on public-facing pages
- Use display names or obfuscated identifiers for admin accounts

---

14. Information Disclosure – User Emails via Product Reviews

Attribute	Value
Risk Level	🟢 LOW (3/10)
Exploitation Likelihood	Likely
Business Impact	Minor
Remediation Difficulty	Easy

Security Implications

Multiple user email addresses are exposed publicly in product reviews, enabling targeted phishing, account enumeration, and credential stuffing attacks.

Steps to Reproduce

1. Browse product pages — no authentication required
2. **Green Smoothie** review → `jim@juice-sh.op`
3. **Banana Juice** review → `bender@juice-sh.op`
4. **Permafrost 2020 Edition** page → `mc.safesearch@juice-sh.op`

All email addresses were accessible to unauthenticated users.

Recommendations

- Replace email addresses with display names or usernames in publicly visible reviews
- Audit all public-facing pages for unintended user data exposure

---

15. Missing Access Control – Unauthorized Deletion of Customer Reviews

Attribute	Value
Risk Level	🟢 LOW (2/10)
Exploitation Likelihood	Possible

Attribute	Value
Business Impact	Minor
Remediation Difficulty	Easy

Security Implications

The admin panel allows deletion of all customer reviews including 5-star feedback without any audit trail, which could be abused to manipulate reputation or suppress legitimate user feedback.

Steps to Reproduce

1. Log in as admin
2. Navigate to `http://10.114.140.87/#/administration`
3. All customer reviews are visible with a delete (bin) icon next to each
4. Click the bin icon next to any 5-star review
5. The review is permanently deleted with no confirmation dialog or activity log

Flag: `78231b75c0b2180b7e964dcbb1ab3c3f58639f2e`

Recommendations

- Implement an audit log for all administrative actions including content deletion
- Add a confirmation step and require a reason before deleting user-generated content

---

Flags Summary

Challenge	Flag
Login Admin (SQL Injection)	`690fa3247a99d651e0b26f947baf0b79b4f404a9`
Login Bender (SQL Injection)	`5ff5052e879e6fef64124e64c82c84ebc809c6c4`
Password Strength (admin123)	`ff4aebffe31b0ffdea9bdd0207a16a3c01ac6c56`
Reset Jim's Password	`3c3e2d6ef99b733b947e92f8e2a9ed08bf57ea63`
Confidential Document (/ftp/)	`8d2072c6b0a455608ca1a293dc0c9579883fc6a5`
Login MC SafeSearch	`bb105418e73708ceccf1a7b2491f434b8f5230e4`
Error Handling	`9c297196ecf8890bc1e900fcf3aebae8c9f9880a`
Forgotten Developer Backup	`dad737329153068527159005db4eb139e86e76ba`

Challenge	Flag
Poison Null Byte	`cfdeea14e8f01b4952722fd0e4a77f1928593c9a`
Admin Section	`71aeb3b0bf01cc6e488f0207bb62f79b41454a87`
View Basket (IDOR)	`e6982b34b6734ceadd28e5019b251f929a80b815`
Five-Star Feedback Deletion	`78231b75c0b2180b7e964dcbb1ab3c3f58639f2e`
DOM XSS	`4a31a4fe0954199566e360a873802bf64d0d0a84`
HTTP-Header XSS (Persistent)	`c37da14686b69a220fd9febd09bb9593e7d0539f`
Reflected XSS	`305021787d3e9cd9cebc057a021c2504550bb3b6`
Score Board	`2614339936e8282e2f820f023d4d998a1f95e02a`

Tools Used

Tool	Purpose
Burp Suite Community Edition	HTTP proxy for intercepting, modifying, and replaying web requests. Used for all injection and XSS testing.
Firefox DevTools (Debugger)	Used to inspect bundled JavaScript files and discover hidden application routes.
Web Browser	Manual application walking, reconnaissance, and confirming exploits.
Google / OSINT	Research security question answers (Star Trek replicator reference for Jim's account).

This writeup was produced from a TryHackMe OWASP Juice Shop lab assessment. All testing was performed in a controlled, authorized environment.